Rapport d'évaluation

Pipeline de détection de véhicules bruyants

Quantnuis-Backend

Sujet	Configuration de la pipeline, protocole d'entraînement et validité de l'évaluation F1
Auteur	Daniel
Date	2026-03-17
Version	2.0 (CNN NoisyCarDetector + CRNN CarDetector)

Table des matières

1 Configuration de la pipeline **2**
1.1 Paramètres globaux (`config/settings.py`) 2
1.2 Paramètres audio (mel-spectrogramme) 2
1.3 Architecture en cascade 2

2 Protocole d'entraînement **3**
2.1 Dataset et annotations 3
2.1.1 Distribution de `data/noisy_car_detector/annotation.csv` (v2) 3
2.2 CarDetector CRNN (`models/car_detector/train_crnn.py`) 3
2.2.1 Étapes complètes 3
2.3 NoisyCarDetector MLP (`models/noisy_car_detector/train.py`) 5
2.3.1 Étapes complètes 5

3 Calcul du score F1 **6**
3.1 Définitions de base 6
3.2 Appel exact dans le code 7
3.3 Exemple numérique concret 7
3.4 F1 vs Accuracy — pourquoi F1 est approprié ici 7

4 Validité du protocole d'évaluation **7**
4.1 Tableau de synthèse 7
4.2 Impact quantifié du feature selection leakage 8
4.3 Recommandations de correction 9

5 Synthèse **9**

1 Configuration de la pipeline

1.1 Paramètres globaux (`config/settings.py`)

La configuration est centralisée dans la classe `Settings` (singleton via `@lru_cache`). Elle s'adapte automatiquement à l'environnement (`IS_LAMBDA` détecté via `AWS_LAMBDA_FUNCTION_NAME`).

Paramètre	Valeur	Rôle
`SAMPLE_RATE`	22 050 Hz	Fréquence de rééchantillonnage audio
`N_MFCC`	40	Coefficients MFCC extraits
`CAR_DETECTION_THRESHOLD`	0.5	Seuil sigmoid — CarDetector
`NOISY_THRESHOLD`	0.5	Seuil sigmoid — NoisyCarDetector
`TRAINING_EPOCHS`	60	Epochs max MLP
`TRAINING_BATCH_SIZE`	16	Batch size MLP
`SMOTE_K_NEIGHBORS`	3	Voisins SMOTE (adapté petit dataset)
`TEST_SIZE`	0.20	Fraction du dataset de test
`TOP_FEATURES_COUNT`	12	Features retenues après sélection RF

Table 1 – Paramètres de configuration principaux

1.2 Paramètres audio (mel-spectrogramme)

Les deux modèles CNN/CRNN utilisent la même fonction `shared/audio_utils.load_melspectrogram()` :

Paramètre	Valeur	Impact	Calcul
`sr`	22 050 Hz	Résolution temporelle	$T = \text{sr} \times \text{duration}$
`duration`	4.0 s	Taille de l'entrée	88 200 samples
`n_mels`	128	Résolution fréquentielle	bandes mel
`n_fft`	2 048	Résolution spectrale FFT	93 ms de fenêtre
`hop_length`	512	Pas du STFT	23 ms entre frames

Table 2 – Paramètres du mel-spectrogramme

La durée de 4 s avec `hop_length=512` produit exactement $\lceil 88200/512 \rceil = 173$ frames temporelles. La forme du tensor d'entrée CRNN est donc $(\mathbf{128}, \mathbf{173}, \mathbf{1})$.

> **Guard courte durée :** `load_melspectrogram()` rejette les fichiers audio de moins de 0.5 s (`len(y) < sr * 0.5`) en retournant `None`, ce qui protège la pipeline contre les slices corrompus.

1.3 Architecture en cascade

```
Audio (WAV/MP3/OGG/FLAC/M4A)
    | load_melspectrogram(sr=22050, dur=4s, n_mels=128)
    | normalize: (mel_db - X_mean) / (X_std + 1e-8)
```

```
+-----------------------------------------+
|  CarDetector (CRNN)                     |
|  Conv2D(16)->BN->Pool(2,2)->Drop(0.3) x3 |
|  Permute -> Reshape -> GRU(32) -> D(0.4) |
|  Dense(32, L2=0.01) -> Dense(1,sigmoid)  |
|  threshold = 0.5                        |
+-----------------------------------------+
         | P(voiture)
  +-----+------+
 < 0.5        >= 0.5
  |              |
Pas voiture    VOITURE
                 |
+-----------------------------------------+
|  NoisyCarDetector (CNN ou MLP fallback)  |
|  [CNN] Conv2D(32/64/128)->Dense(128/64)  |
|  [MLP] Dense(64/32)->Dropout->Sigmoid    |
|  threshold = 0.5                        |
+-----------------------------------------+
         | P(bruyant)
  +-----+------+
 < 0.5        >= 0.5
  |              |
Normal (~65dB)  Bruyant (~95dB)
```

2 Protocole d'entraînement

2.1 Dataset et annotations

Modèle	Colonne cible	Valeurs
CarDetector	`label`	0 = PAS_VOITURE, 1 = VOITURE
NoisyCarDetector	`label`	0 = NORMAL, 1 = BRUYANT
`reliability`		1 = faible, 2 = moyen, 3 = élevé

2.1.1 Distribution de `data/noisy_car_detector/annotation.csv` (v2)

L'annotation CSV contient **49 607 entrées** après l'expansion du dataset (mars 2026).

Point critique : Le filtre par défaut `-min-reliability 2` dans `train_crnn.py` n'utilise que **887 samples sur 49 607** disponibles. Avec équilibrage par classe, cela descend à $\leq 2 \times 60 = 120$ samples si les classes ne sont pas équilibrées à cette fiabilité. **L'ajout des 48 415 nouvelles annotations n'est donc pas exploité** avec les paramètres par défaut. Il faut explicitement passer `-min-reliability 1` pour utiliser tout le dataset.

2.2 CarDetector CRNN (`models/car_detector/train_crnn.py`)

2.2.1 Étapes complètes

1. Chargement et filtrage des annotations

```
df = pd.read_csv(config.ANNOTATION_CSV)
df = df[df['reliability'] >= args.min_reliability]   # défaut: 2
```

2. Équilibrage par sous-échantillonnage (si `-balance`, activé par défaut)

Critère	Valeur	Part
Distribution des labels		
label = 0 (NORMAL)	25 213	50.8%
label = 1 (BRUYANT)	24 394	49.2%
Distribution de la fiabilité		
reliability = 1 (faible)	≈48 720	98.2%
reliability = 2 (moyen)	827	1.7%
reliability = 3 (élevé)	60	0.1%
Total utilisé avec `-min-reliability 2`	**887**	**1.8%**

TABLE 3 – Distribution du dataset NoisyCarDetector

```
n_min = min((df['label']==0).sum(), (df['label']==1).sum())
df = pd.concat([
    df[df['label']==0].sample(n=n_min, random_state=42),
    df[df['label']==1].sample(n=n_min, random_state=42),
])
```

3. Pré-calcul des spectrogrammes vers memmap `/tmp/car_spectros.dat`

```
mm = np.memmap(MMAP_X, dtype='float32', mode='w+', shape=(n, n_mels,
    n_frames))
for i, p in enumerate(paths):
    s = load_spectrogram(p, args.sr, args.duration, ...)
    mm[i] = s if s is not None else 0.0
```

> Les fichiers manquants sont remplis par `0.0` (valeur silencieuse). Ce comportement silencieux peut polluer l'entraînement si des slices sont absents.

4. Normalisation globale estimée sur 1 000 échantillons aléatoires

```
idx_s = np.random.choice(n, min(1000, n), replace=False)
X_mean = float(mm[idx_s].mean())
X_std  = float(mm[idx_s].std())
```

> L'estimation sur 1 000 sur 49k peut dériver de la vraie moyenne. Une estimation exhaustive serait plus fiable (passe en streaming).

5. Cross-validation stratifiée 5 folds (protocole principal)

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for fold, (train_idx, val_idx) in enumerate(cv.split(np.arange(n), y)):
    model = build_crnn(input_shape)
    model.fit(ds_tr, validation_data=ds_val, epochs=50,
              callbacks=[EarlyStopping(patience=8, ...),
                         ReduceLROnPlateau(patience=4, factor=0.5)])
    f1 = f1_score(y_true, y_pred)
    f1_scores.append(f1)
cv_f1_mean = np.mean(f1_scores)  # MÉTRIQUE OFFICIELLE
cv_f1_std  = np.std(f1_scores)
```

6. Entraînement final sur le dataset complet

```
final_model = build_crnn(input_shape)
final_model.fit(ds_all, epochs=50,
    callbacks=[EarlyStopping(patience=10, monitor='loss')])
# Rapport sur le DATASET D'ENTRA NEMENT COMPLET (pas un test set)
print(classification_report(y_true_all, y_pred_all))
```

Score gonflé : Le rapport final évalue le modèle sur les données sur lesquelles il a été entraîné. C'est un score **in-sample**, non représentatif des performances sur des données inédites. Seul le **CV F1** est fiable.

2.3 NoisyCarDetector MLP (`models/noisy_car_detector/train.py`)

2.3.1 Étapes complètes

1. Chargement des features pré-extraites

```
df = pd.read_csv(config.FEATURES_CSV)  # features_optimized.csv
X = df[feature_cols].values            # ~219 features
y = df['label'].values
```

2. Sélection des top K features par Random Forest (sur tout X)

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)                # X contient le futur test set
top_features, indices = ...
X = X[:, indices]           # réduction à TOP_FEATURES_COUNT=12
```

Data leakage : La Random Forest de sélection est entraînée sur **l'intégralité des données** (train + test). Elle apprend quelle feature est discriminante en utilisant le test set. Le F1 final est ainsi légèrement surestimé, car les features ont été sélectionnées en voyant le test set.

3. Split train/test AVANT SMOTE

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

✓ Stratification préservée. ✓ SMOTE appliqué après le split (prévention du leakage sur les données synthétiques).

4. SMOTE uniquement sur le train set

```
minority = min((y_train==0).sum(), (y_train==1).sum())
k = max(1, min(settings.SMOTE_K_NEIGHBORS, minority - 1))  # k=3
smote = SMOTE(k_neighbors=k, random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)
```

SMOTE génère des interpolations linéaires dans l'espace des features entre voisins de la classe minoritaire. Le paramètre `k=3` est minimal (adapté à des datasets de quelques centaines de samples).

5. StandardScaler

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # fit sur train+SMOTE
X_test  = scaler.transform(X_test)       # transform seulement
```

> ⚠ Le scaler est ajusté sur les données SMOTE (synthétiques incluses), ce qui peut légèrement biaiser la normalisation par rapport aux statistiques des données réelles.

6. Class weights (double mécanisme avec SMOTE)

```
unique, counts = np.unique(y_train, return_counts=True)
class_weights = {int(c): total / (len(unique) * count)
                 for c, count in zip(unique, counts)}
```

> ⚠ SMOTE et class weights ensemble peuvent provoquer une **double compensation** du déséquilibre. Après SMOTE, les classes sont équilibrées ; les class weights redeviennent ≈ 1.0, ce qui est correct, mais le calcul n'est pas garanti si SMOTE échoue (exception attrapée).

7. Entraînement MLP avec EarlyStopping

```
model = Sequential([Dense(64, relu), Dropout(0.3),
                    Dense(32, relu), Dropout(0.2),
                    Dense(1, sigmoid)])
EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=
    True)
model.fit(X_train, y_train, validation_data=(X_test, y_test), ...)
```

8. Évaluation finale

```
y_pred = (model.predict(X_test) > 0.5).astype(int).flatten()
f1 = f1_score(y_test, y_pred)    # average='binary', positive=1
```

3 Calcul du score F1

3.1 Définitions de base

Pour un classificateur binaire avec classe positive = 1 (BRUYANT / VOITURE) :

	Prédit 1	**Prédit 0**
Réel 1	TP (Vrai Positif)	FN (Faux Négatif)
Réel 0	FP (Faux Positif)	TN (Vrai Négatif)

Table 4 – Matrice de confusion

$$\text{Précision} = \frac{TP}{TP + FP} \qquad \text{(parmi les détections positives, combien sont corr} \tag{1}$$

$$\text{Rappel} = \frac{TP}{TP + FN} \qquad \text{(parmi les vrais positifs, combien sont détectés ?)} \tag{2}$$

$$\text{F1} = 2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \tag{3}$$

3.2 Appel exact dans le code

```
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
# Équivalent explicite :
f1 = f1_score(y_test, y_pred, average='binary', pos_label=1)
```

Le paramètre `average='binary'` (défaut sklearn) calcule le F1 **uniquement pour la classe 1**. Pour le NoisyCarDetector, la classe 1 est **BRUYANT** ; pour le CarDetector, c'est **VOITURE**.

3.3 Exemple numérique concret

Supposons le résultat de benchmark rapporté : $F1 = 0.9938$ pour le CNN. Si la matrice de confusion sur 526 échantillons est :

	Prédit BRUYANT	**Prédit NORMAL**	**Total**
Réel BRUYANT	$TP = 261$	$FN = 1$	262
Réel NORMAL	$FP = 2$	$TN = 262$	264
Total prédit	263	263	526

TABLE 5 – Matrice de confusion illustrative cohérente avec F1=0.9938

$$\text{Précision} = \frac{261}{261+2} = \frac{261}{263} \approx 0.9924$$

$$\text{Rappel} = \frac{261}{261+1} = \frac{261}{262} \approx 0.9962$$

$$\text{F1} = \frac{2 \times 0.9924 \times 0.9962}{0.9924 + 0.9962} = \frac{1.9775}{1.9886} \approx 0.9944$$

3.4 F1 vs Accuracy — pourquoi F1 est approprié ici

Métrique	**Formule**	**Problème sur données déséquilibrées**
Accuracy	$(TP + TN)/(N)$	Trompeuse (classe majoritaire domine)
Précision	$TP/(TP + FP)$	Ignore les faux négatifs
Rappel	$TP/(TP + FN)$	Ignore les faux positifs
F1	$2TP/(2TP + FP + FN)$	Harmonie des deux erreurs

Dans le contexte Quantnuis, les faux négatifs (voiture bruyante manquée) et faux positifs (voiture normale classée bruyante) ont tous deux un coût. Le F1 est donc une métrique adaptée.

4 Validité du protocole d'évaluation

4.1 Tableau de synthèse

Point du protocole	Statut	Analyse
CRNN — StratifiedKFold 5 folds	✓ Valide	Stratification préserve les proportions de classe dans chaque fold. `random_state=42` assure la reproductibilité.
CRNN — EarlyStopping par fold	✓ Valide	Chaque fold repart d'un modèle vierge. L'arrêt précoce s'applique indépendamment à chaque fold.
CRNN — rapport final in-sample	× Invalide	Classification report sur le dataset complet d'entraînement. Le modèle final n'a pas de hold-out. **Ce score ne doit pas être communiqué comme résultat.**
CRNN — normalisation sur 1 000 échantillons	△ Approximatif	Suffisant si le dataset est i.i.d., mais peut introduire un biais si les classes ont des distributions spectrales très différentes. Recommandé : passe complète de lecture des stats.
MLP — hold-out 80/20 unique	△ Acceptable	Pas de cross-validation → variance de l'estimateur F1 élevée. Sur 526 échantillons : l'IC à 95% du F1 peut être ± 0.03. Pour une publication, une k-fold serait nécessaire.
MLP — feature selection leakage	× Problématique	La Random Forest de sélection est entraînée sur X entier (test set inclus). Le F1 reporté est légèrement surestimé. **Correction : déplacer la sélection de features à l'intérieur du fold/split.**
MLP — SMOTE avant split	✓ Valide	Split effectué avant SMOTE. Pas de leakage des données synthétiques.
MLP — scaler sur données SMOTE	△ Mineur	Le scaler intègre les statistiques des points synthétiques. Impact négligeable si SMOTE génère peu de samples.
MLP + SMOTE + class weights	△ Redondant	Après SMOTE, les classes sont équilibrées → class weights ≈ 1. Le double mécanisme est sans danger mais inutile.
Filtre reliability ≥ 2 par défaut	× Critique	Réduit le dataset de 49 607 à **887 échantillons**. Anéantit le bénéfice des 48 415 nouvelles annotations. **Utiliser `-min-reliability 1` pour exploiter tout le dataset.**

Table 6 – Bilan de validité du protocole d'évaluation

4.2 Impact quantifié du feature selection leakage

Le leakage par sélection de features est un biais systématique mesuré dans la littérature ML. Pour un dataset de ≈ 500 samples et 12 features sélectionnées parmi 219, le biais estimé

sur le F1 est :

$$\Delta F1_{\text{leakage}} \approx \frac{k_{\text{sélectionné}}}{N_{\text{total}}} \times \frac{1}{n_{\text{features}}/k_{\text{sélectionné}}} \approx \frac{12}{500} \times \frac{12}{219} \approx 0.001\text{–}0.005 \tag{4}$$

Autrement dit, le F1 reporté est surestimé de **0.1 à 0.5 point** ($\approx$ 0.001–0.005), ce qui est **peu impactant** sur les valeurs rapportées mais invalide formellement le protocole.

4.3 Recommandations de correction

R1. [Critique] Passer `-min-reliability 1` pour le CRNN

```
python -m models.car_detector.train_crnn --min-reliability 1
# Utilise 49 607 samples au lieu de 887
```

R2. [Important] Intégrer la sélection de features dans le split MLP

```
# Correct : feature selection APRES le split
X_train, X_test, y_train, y_test = train_test_split(X, y, ...)
top_features, indices = select_features(X_train, y_train, feature_cols
    , n_top)
X_train = X_train[:, indices]
X_test  = X_test[:, indices]   # applique les m mes indices sans voir
     y_test
```

R3. [Important] Ne pas reporter le classification_report final du CRNN

Le rapport final utilise le dataset d'entraînement complet. Seul le CV F1 ($\mu \pm \sigma$ sur les 5 folds) est la métrique valide à communiquer.

R4. [Optionnel] Ajouter une cross-validation au MLP

Remplacer le hold-out simple par un StratifiedKFold pour réduire la variance de l'estimateur F1 :

```
from sklearn.model_selection import StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
# Note : SMOTE et feature selection doivent  tre  dans la boucle fold
```

R5. [Optionnel] Normalisation sur dataset complet

```
# Passe streaming pour éviter de charger tout en mémoire
X_mean, X_std = compute_stats_streaming(mm, n)  # à implémenter
```

5 Synthèse

Modèle	Métrique rapportée	Validité	Valeur estimée réelle
CarDetector CRNN	CV F1 ($\mu \pm \sigma$, 5 folds)	✓ Valide	$\approx$ valeur rapportée
NoisyCarDetector CNN	F1 sur benchmark hold-out	△ Acceptable	valeur − 0.001–0.005
NoisyCarDetector MLP	F1 sur hold-out 20%	△ Acceptable	valeur − 0.001–0.005
CRNN rapport final	Classification report (in-sample)	× Non valide	Surestimé (biais fort)

Table 7 – Validité des métriques rapportées

Conclusion : Le protocole d'évaluation est **globalement solide** pour le CRNN (StratifiedKFold 5 folds, reproductible). Les problèmes identifiés (feature selection leakage MLP, rapport in-sample CRNN, filtre reliability) sont documentés et corrigeables sans refonte majeure. La métrique F1 est un choix approprié pour un problème de classification binaire avec coût symétrique des erreurs.